

AIE683 Machine Learning

Week 2: Generalization & Validation

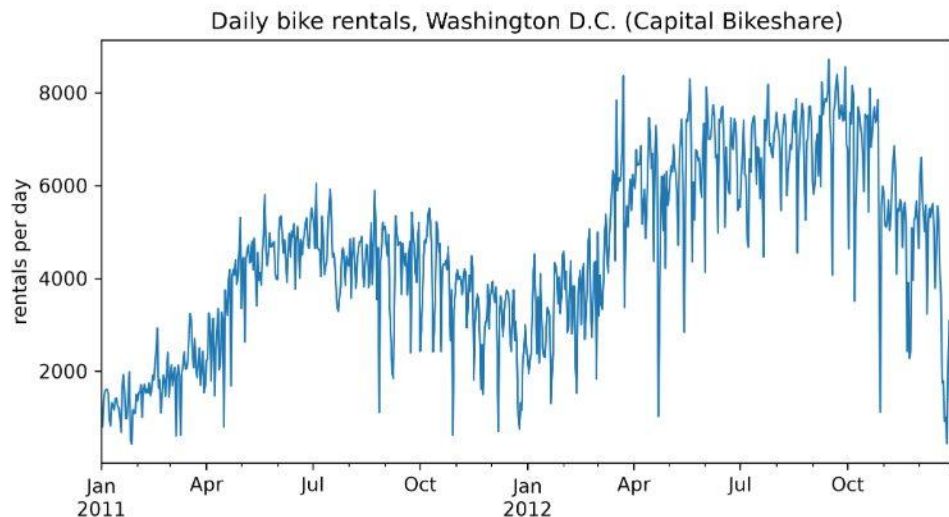
Dr. Hakan Emekci

Hacettepe University, Graduate School of Informatics

A Model That Looked Great

A bike-sharing operator (Capital Bikeshare, Washington D.C.) moves bikes between stations every night. To plan the trucks, it needs **tomorrow's number of rentals**.

- An analyst fits a model on two years of daily rentals, evaluates with a random 80/20 split: $R^2 \approx 0.7$.
- The model goes into production and is evaluated on the following months: the predictions are worse than simply predicting the average.
- Same data, same model. The only difference: **how it was validated**.



Today

- Nearest neighbors as a first supervised model
- Model complexity, overfitting and underfitting, bias and variance
- Train / validation / test splits
- Cross-validation and grid search
- Cross-validation strategies: stratified, repeated, grouped, time series
- Logging a hyper-parameter sweep to MLflow
- Two in-class exercises (Try it 1 and 2) in `lab.ipynb`

Supervised Learning

$(x_i, y_i) \sim p(x, y)$ i.i.d.

$x_i \in \mathbb{R}^p$

$y_i \in \mathbb{R}$

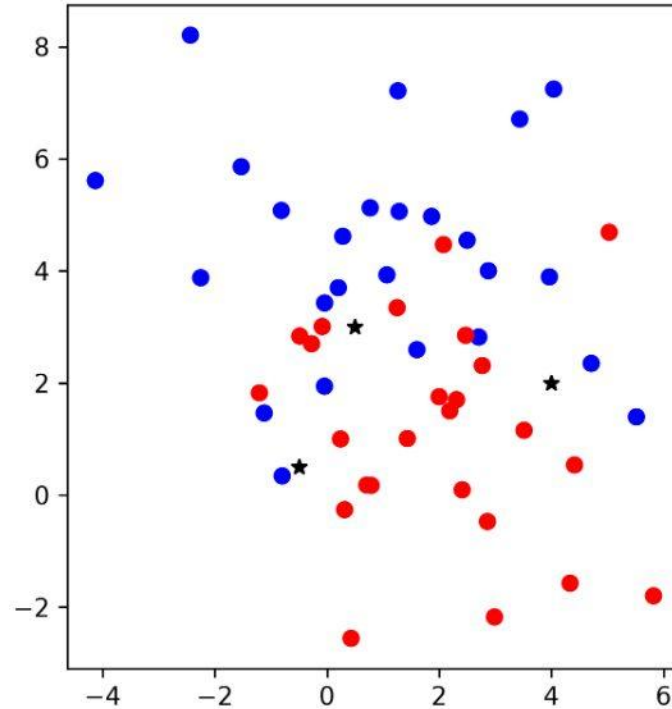
learn $f(x_i) \approx y_i$

such that

$f(x) \approx y$

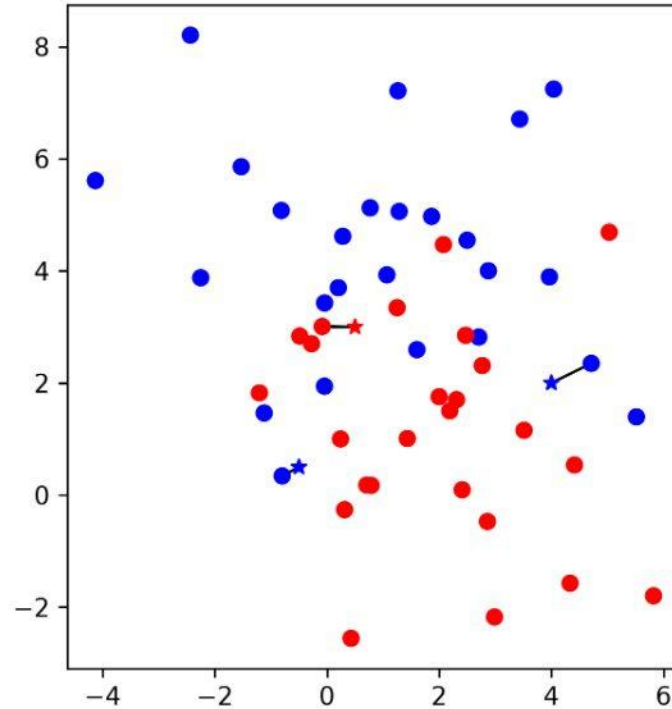
for new $(x, y) \sim p(x, y)$.

Nearest Neighbors



$$f(x) = y_i, i = \operatorname{argmin}_j ||x_j - x||$$

Nearest Neighbors



$$f(x) = y_i, i = \operatorname{argmin}_j ||x_j - x||$$

training set

$$X = \begin{pmatrix} 1.1 & 2.2 \\ 6.7 & 0.5 \\ 2.4 & 9.3 \\ 1.5 & 0.0 \\ 0.5 & 3.5 \\ 5.1 & 9.7 \\ 3.7 & 7.8 \end{pmatrix} \quad y = \begin{pmatrix} 0 \\ 1 \\ 1 \\ 0 \\ 1 \\ 0 \\ 0 \end{pmatrix}$$

test set

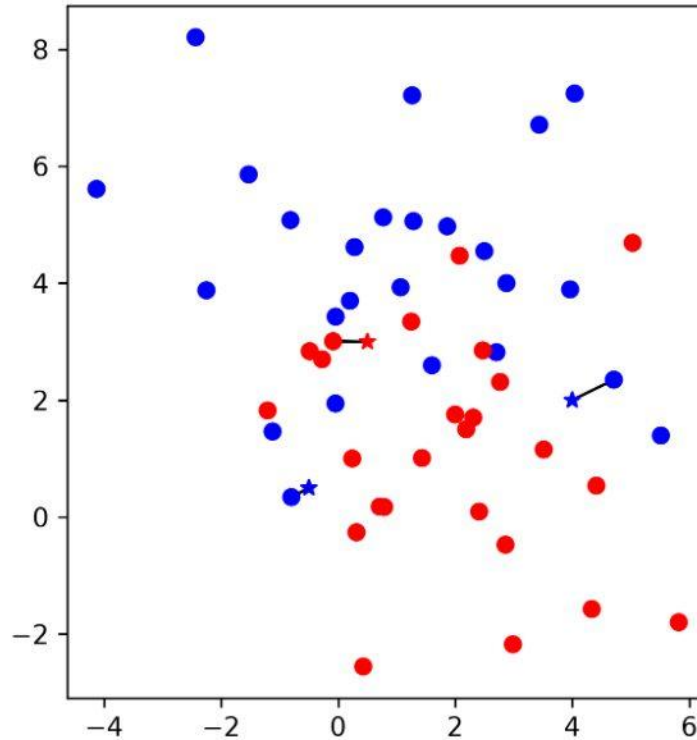
KNN with scikit-learn

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y)

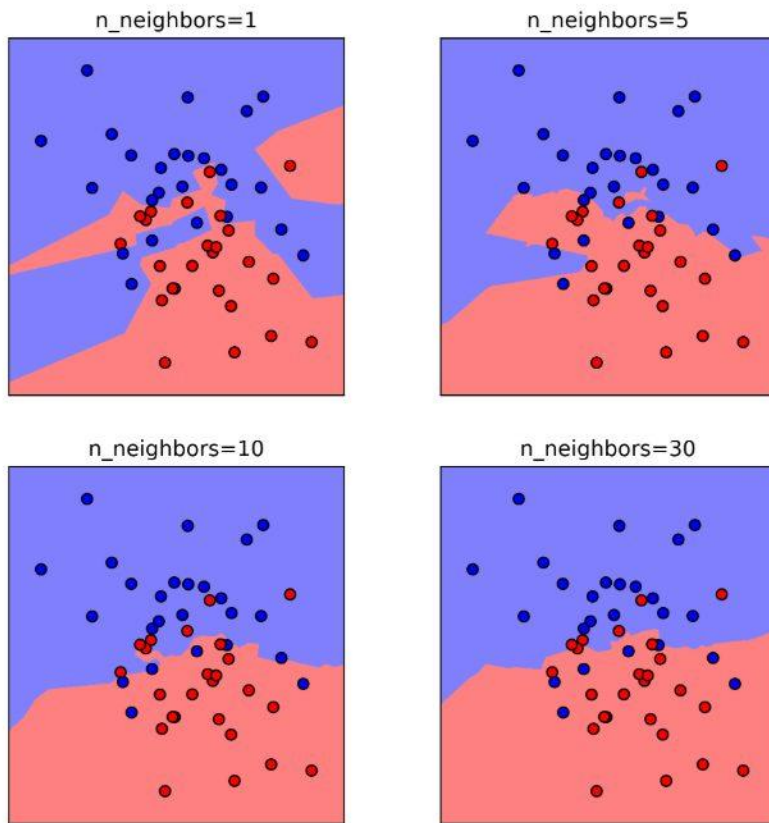
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(X_train, y_train)
print(f"accuracy: {knn.score(X_test, y_test):.2f}")
y_pred = knn.predict(X_test)
```

accuracy: 0.77

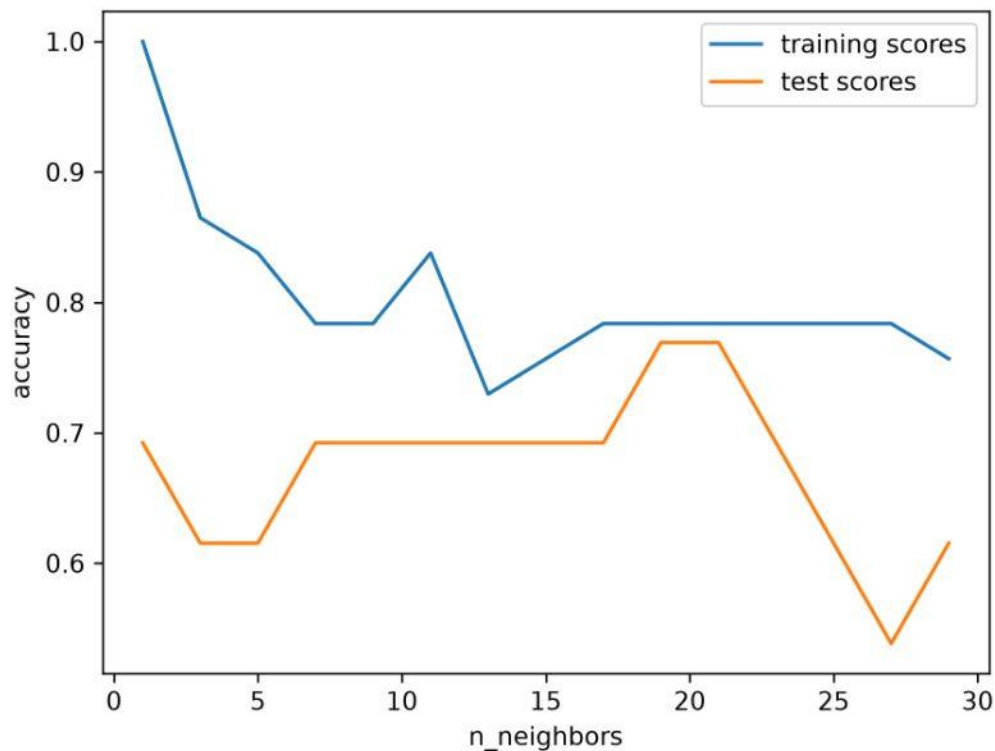
Influence of Number of Neighbors



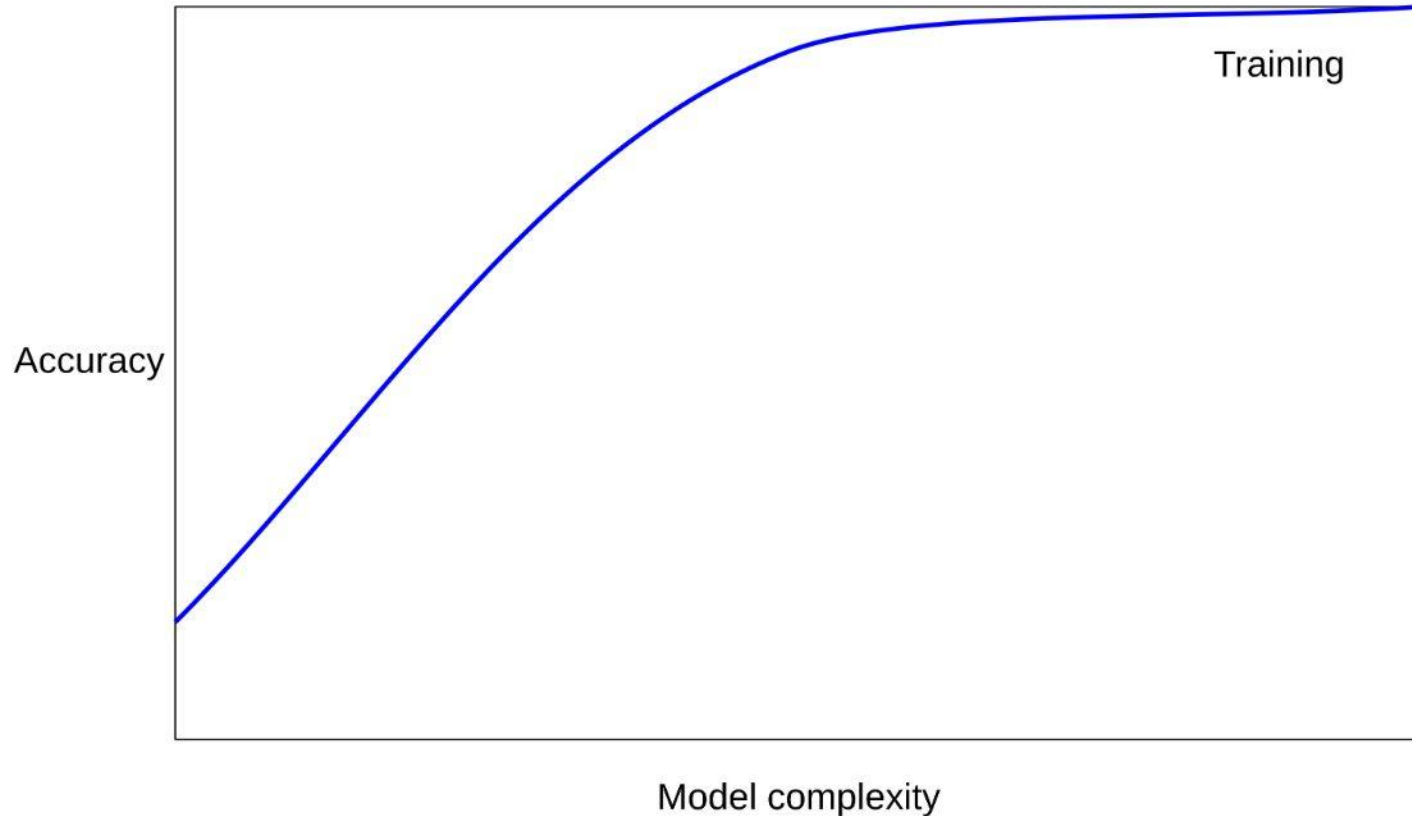
Influence of n_neighbors



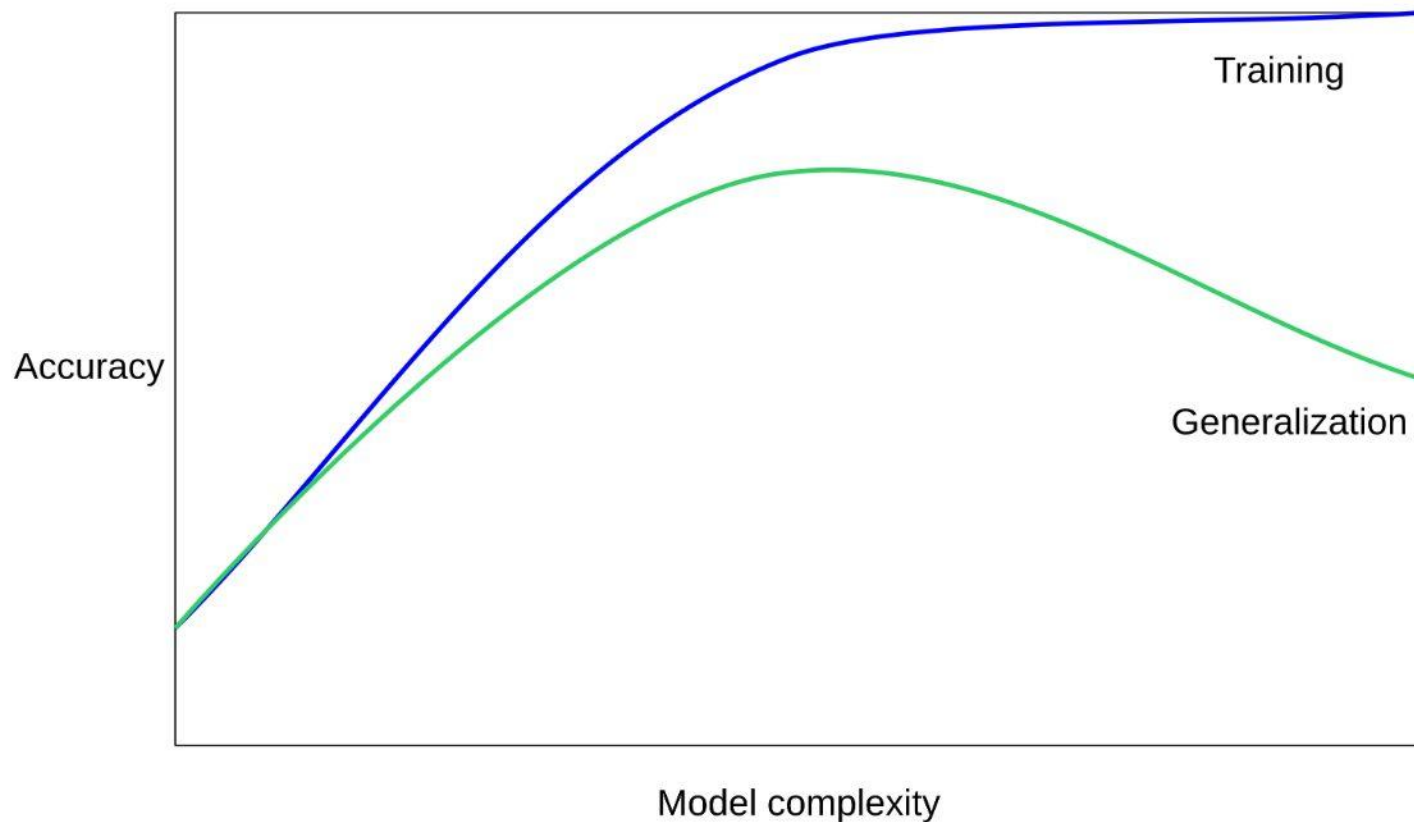
Model complexity



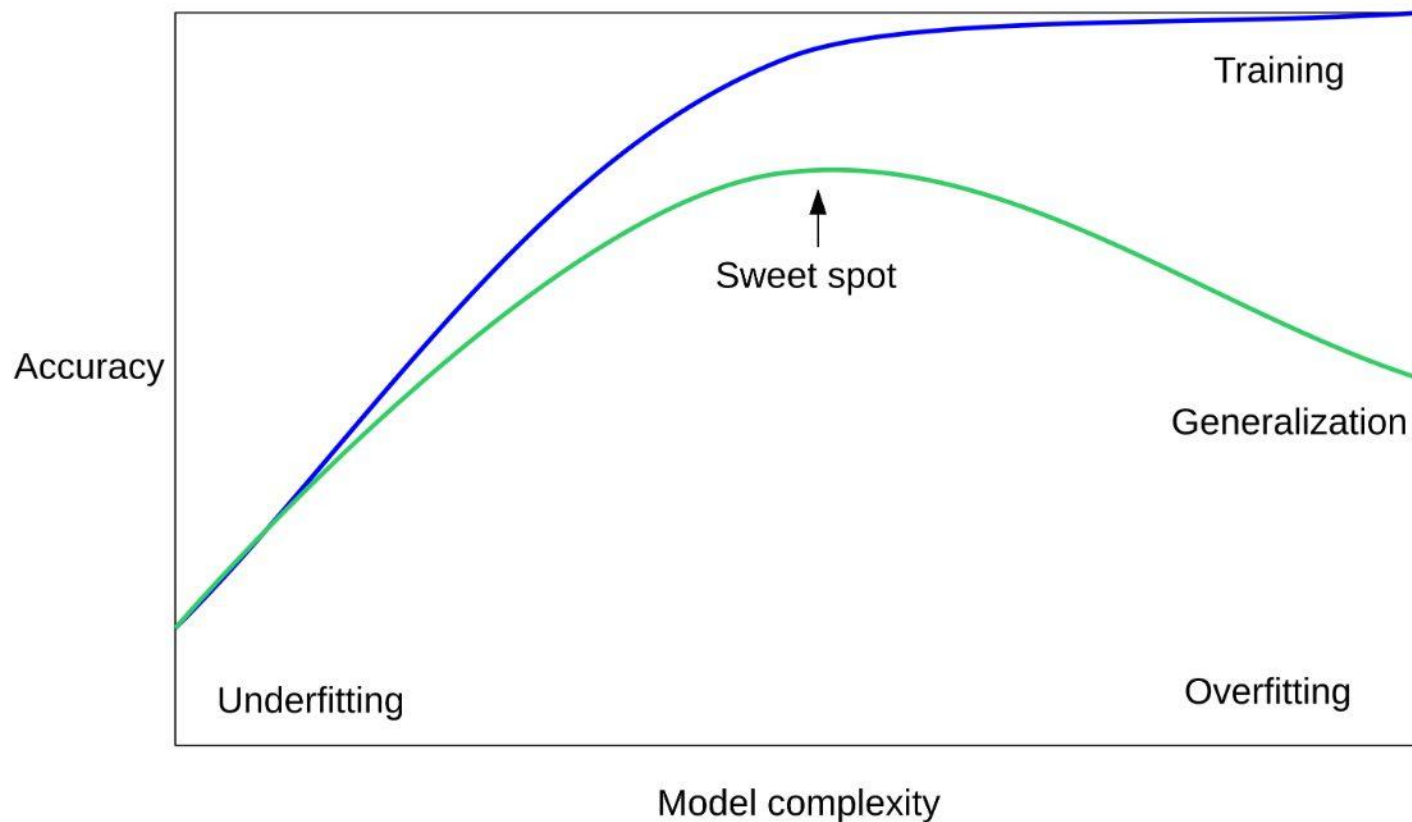
Overfitting and Underfitting



Overfitting and Underfitting



Overfitting and Underfitting



Bias and Variance

For squared loss, with $y = f(x) + \varepsilon$ and $\text{Var}(\varepsilon) = \sigma^2$, the expected error of a model $\hat{f}$ trained on a random training set is

$$\mathbb{E}[(y - \hat{f}(x))^2] = \underbrace{\mathbb{E}(f(x) - \hat{f}(x))^2}_{\text{bias}^2}$$

- $\underbrace{\text{Var}(\hat{f}(x))}_{\text{variance}}$
- $\underbrace{\sigma^2}_{\text{noise}}$
- **Bias**: error of the average model; high for too-simple models (underfitting)
- **Variance**: how much the model changes with the training sample; high for too-complex models (overfitting)
- **Noise**: irreducible, no model can remove it
- kNN: small k gives low bias and high variance; large k gives high bias and low variance

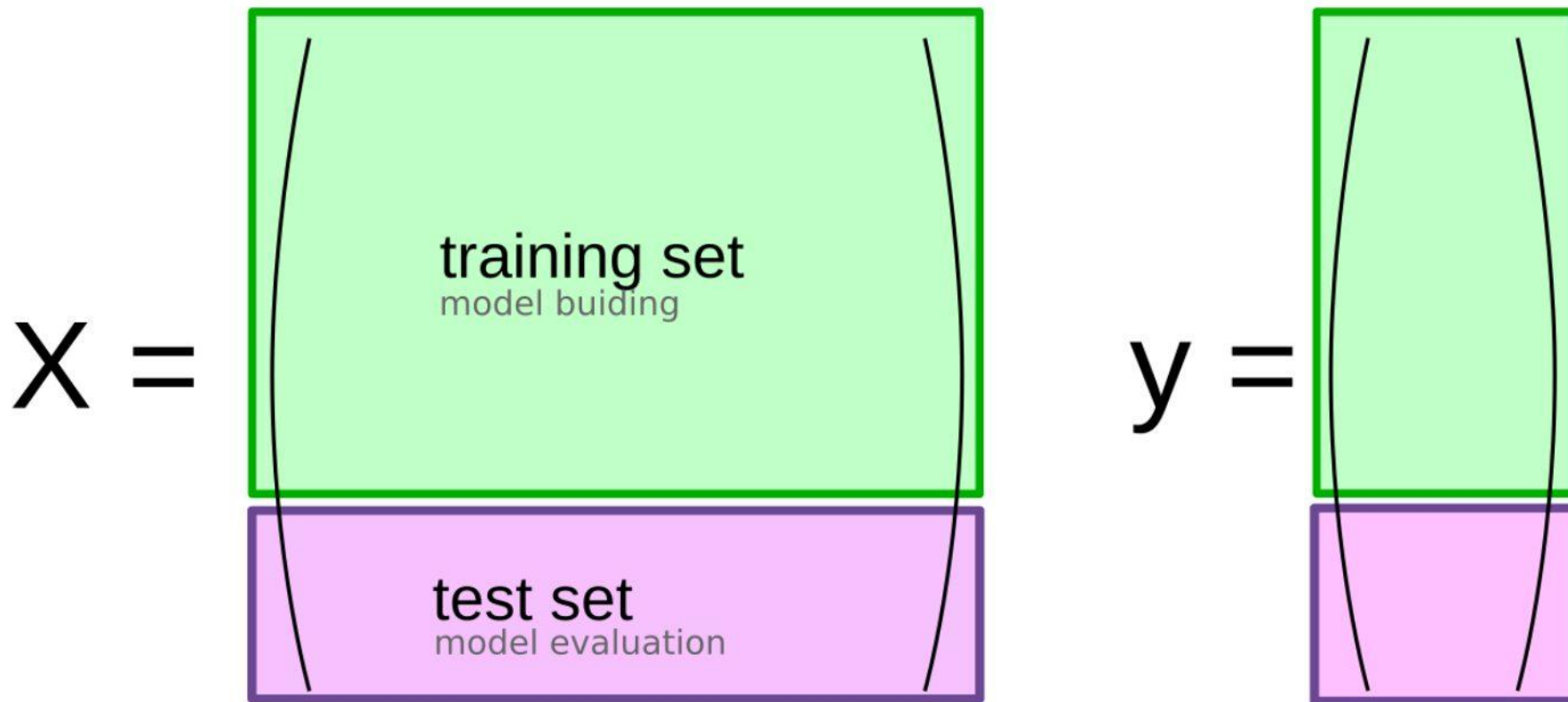
Computational Properties

Naive

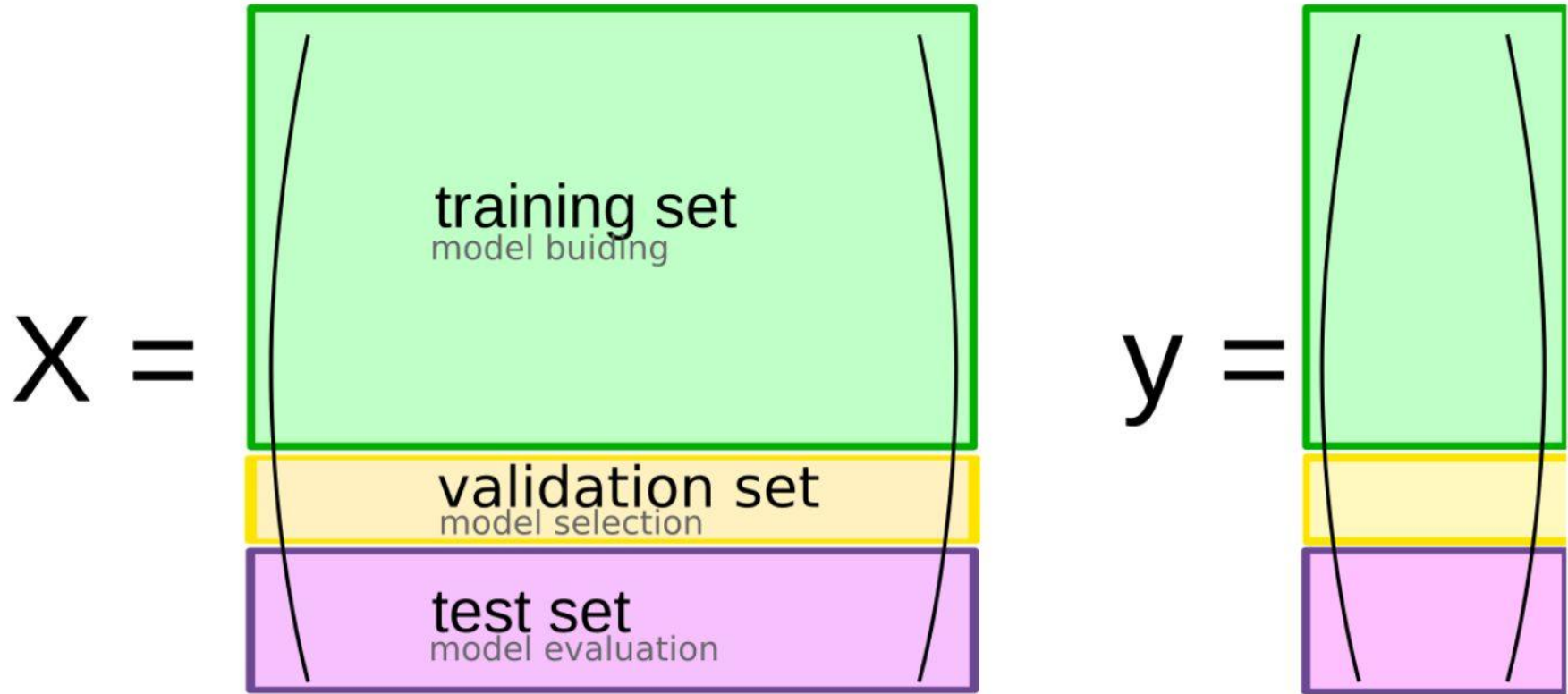
- fit: no time
- memory: $O(n * p)$
- predict: $O(n * p)$

$n=n_{\text{samples}}$ $p=n_{\text{features}}$

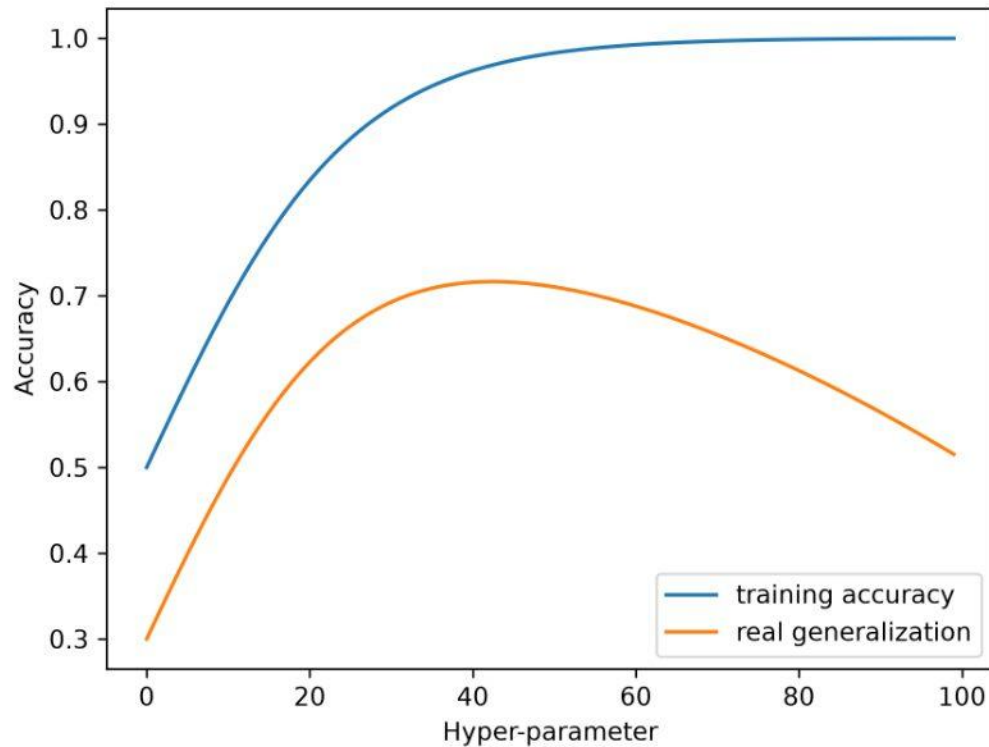
So far: Train-test-split



Threefold split



Overfitting the validation set



Threefold Split for Hyper-Parameters

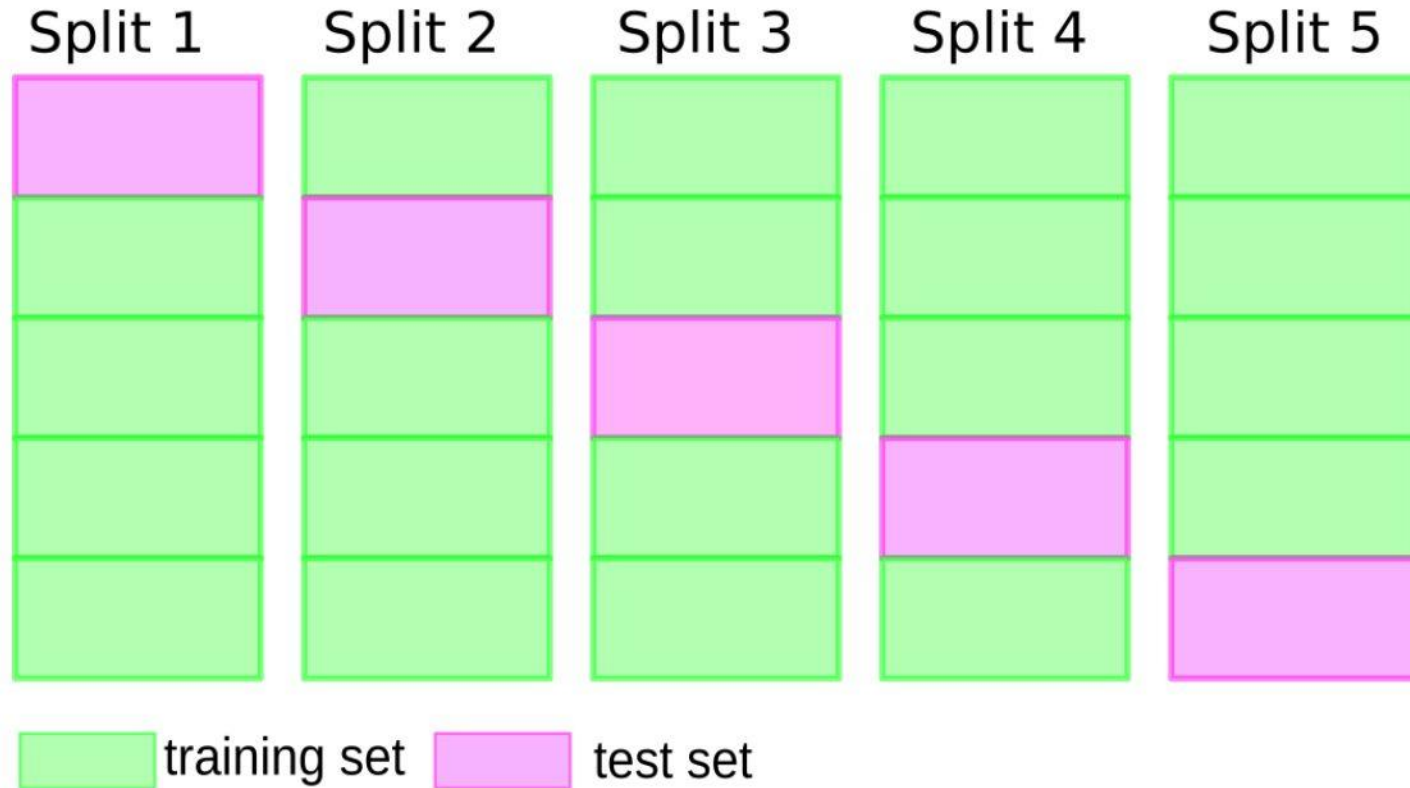
```
X_trainval, X_test, y_trainval, y_test = train_test_split(X, y)
X_train, X_val, y_train, y_val = train_test_split(X_trainval, y_trainval)

val_scores = []
neighbors = np.arange(1, 15, 2)
for i in neighbors:
    knn = KNeighborsClassifier(n_neighbors=i)
    knn.fit(X_train, y_train)
    val_scores.append(knn.score(X_val, y_val))
print(f"best validation score: {np.max(val_scores):.3f}")
best_n_neighbors = neighbors[np.argmax(val_scores)]
print(f"best n_neighbors:", best_n_neighbors)

knn = KNeighborsClassifier(n_neighbors=best_n_neighbors)
knn.fit(X_trainval, y_trainval)
print(f"test-set score: {knn.score(X_test, y_test):.3f}")
```

```
best validation score: 0.925
best n_neighbors: 5
test-set score: 0.937
```

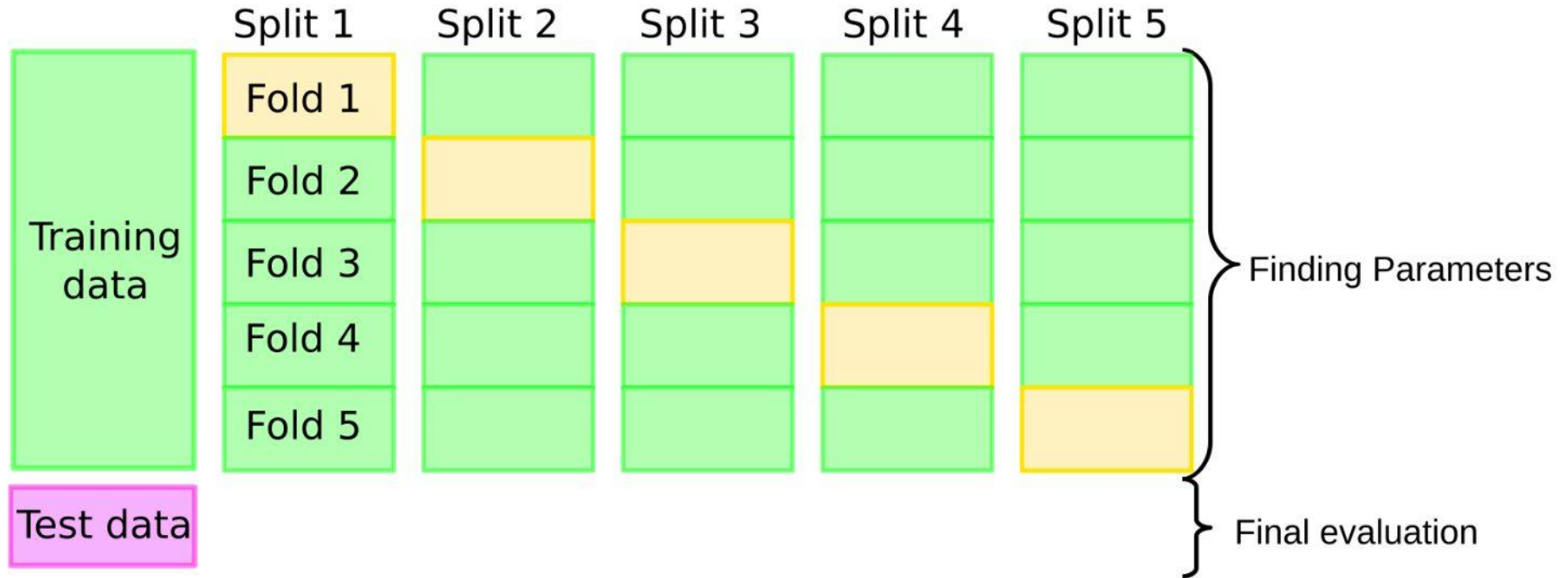
Cross-validation



pro: more stable, more data

con: slower

Cross-validation + test set



Grid-Search with Cross-Validation

```
from sklearn.model_selection import cross_val_score

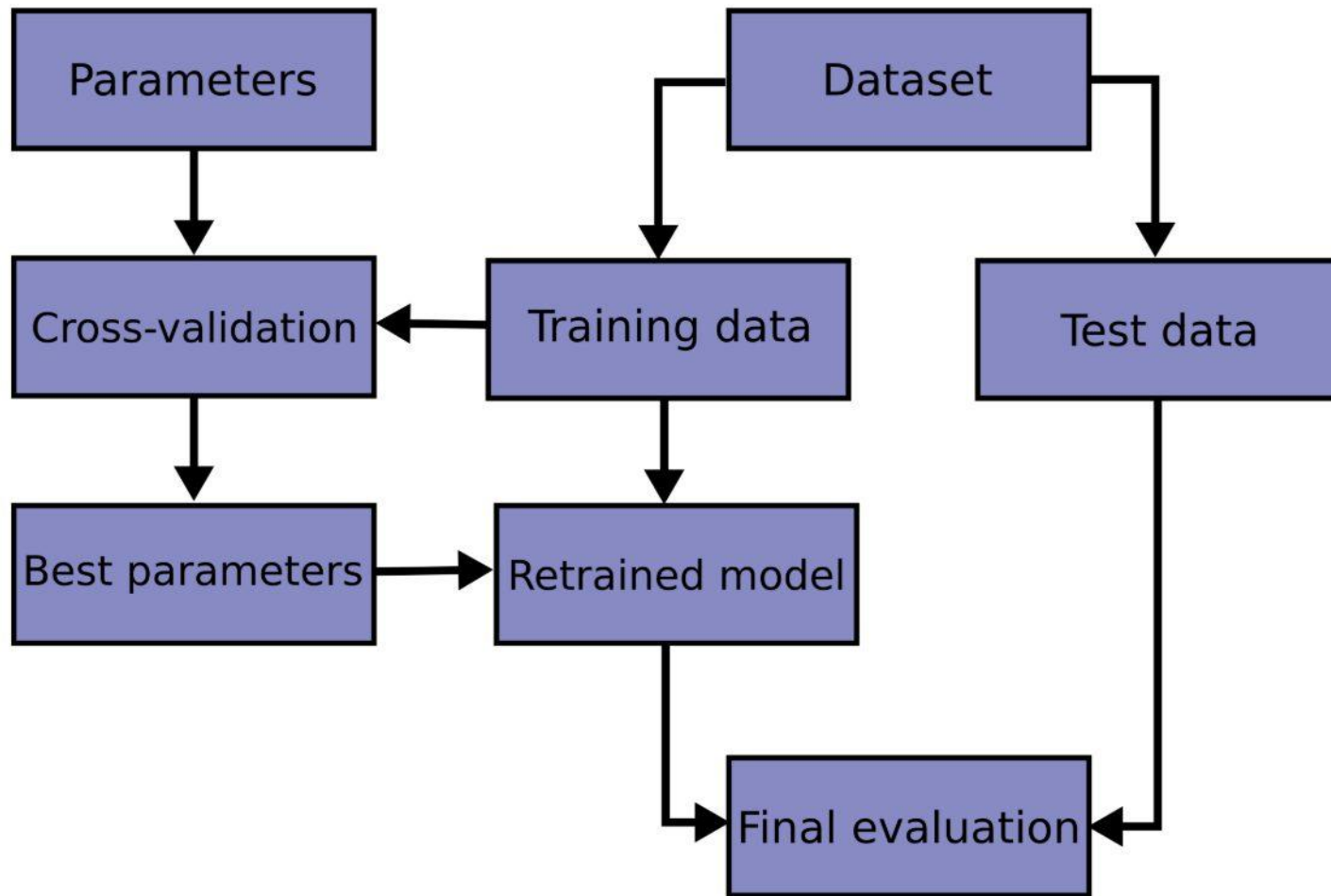
X_train, X_test, y_train, y_test = train_test_split(X, y)

cross_val_scores = []
for i in neighbors:
    knn = KNeighborsClassifier(n_neighbors=i)
    scores = cross_val_score(knn, X_train, y_train, cv=10)
    cross_val_scores.append(np.mean(scores))

print(f"best cross-validation score: {np.max(cross_val_scores):.3f}")
best_n_neighbors = neighbors[np.argmax(cross_val_scores)]
print(f"best n_neighbors: {best_n_neighbors}")

knn = KNeighborsClassifier(n_neighbors=best_n_neighbors)
knn.fit(X_train, y_train)
print(f"test-set score: {knn.score(X_test, y_test):.3f}")
```

```
best cross-validation score: 0.927
best n_neighbors: 7
test-set score: 0.944
```



GridSearchCV

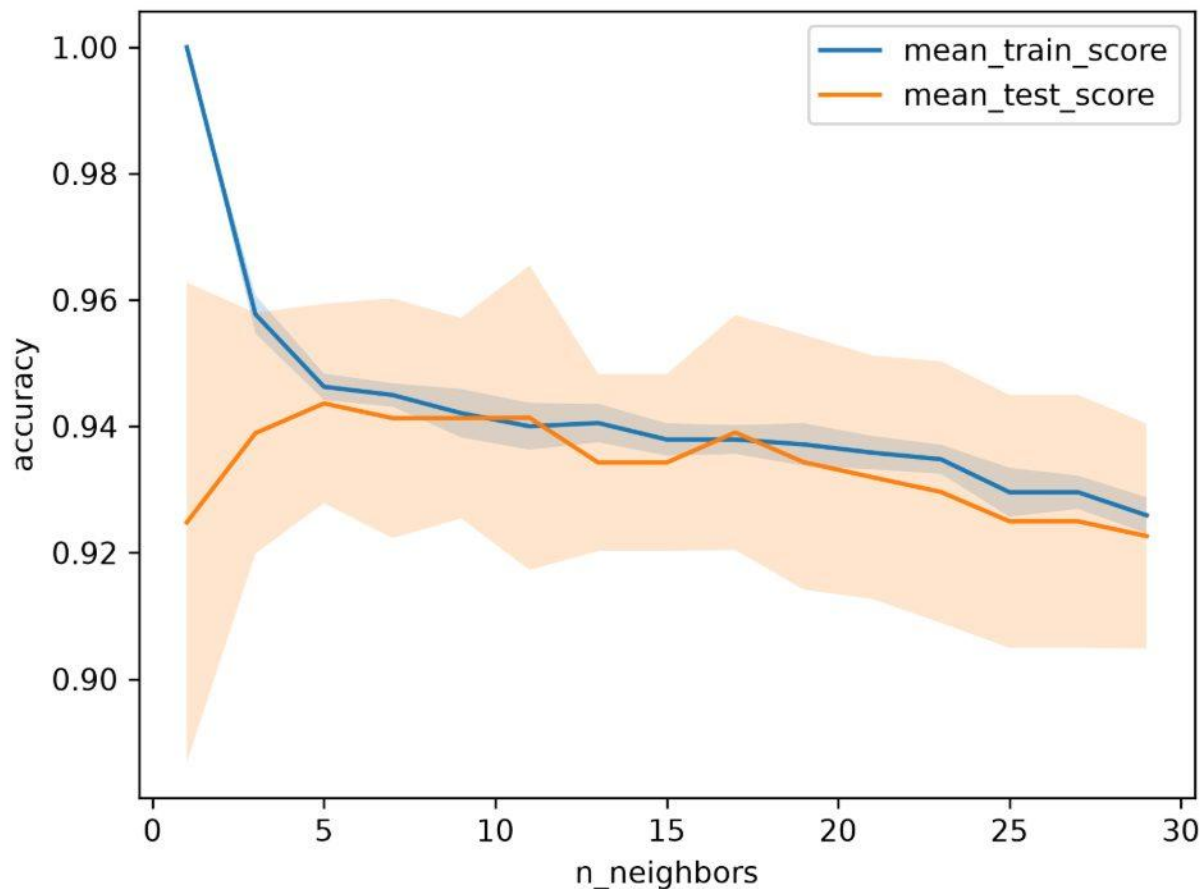
```
from sklearn.model_selection import GridSearchCV

X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y)

param_grid = {'n_neighbors': list(range(1, 30, 2))}
grid = GridSearchCV(KNeighborsClassifier(), param_grid=param_grid, cv=10,
                    return_train_score=True)
grid.fit(X_train, y_train)
print(f"best mean cross-validation score: {grid.best_score_:.3f}")
print(f"best parameters: {grid.best_params_}")
print(f"test-set score: {grid.score(X_test, y_test):.3f}")
```

```
best mean cross-validation score: 0.944
best parameters: {'n_neighbors': 5}
test-set score: 0.916
```

n_neighbors Search Results



Logging the Search to MLflow

```
import mlflow
mlflow.set_experiment("week02-generalization-validation")

cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=0)
with mlflow.start_run(run_name="knn-k-sweep"):
    mlflow.log_params({"dataset": "breast_cancer", "cv": "StratifiedKFold(10, shuffle)"})
    for k in [1, 3, 5, 9, 15, 25]:
        scores = cross_val_score(KNeighborsClassifier(n_neighbors=k),
                                  X_train, y_train, cv=cv)
        with mlflow.start_run(run_name=f"k={k}", nested=True):
            mlflow.log_param("n_neighbors", k)
            mlflow.log_metrics({"cv_accuracy_mean": scores.mean(),
                                "cv_accuracy_std": scores.std()})
            for fold, s in enumerate(scores):
                mlflow.log_metric("cv_accuracy_fold", s, step=fold)
```

Then `uv run mlflow ui` and compare the child runs.

Try it 1: Beat the Held-out Set (15 min, pairs)

lab.ipynb, section "EXERCISE (Try it 1)"

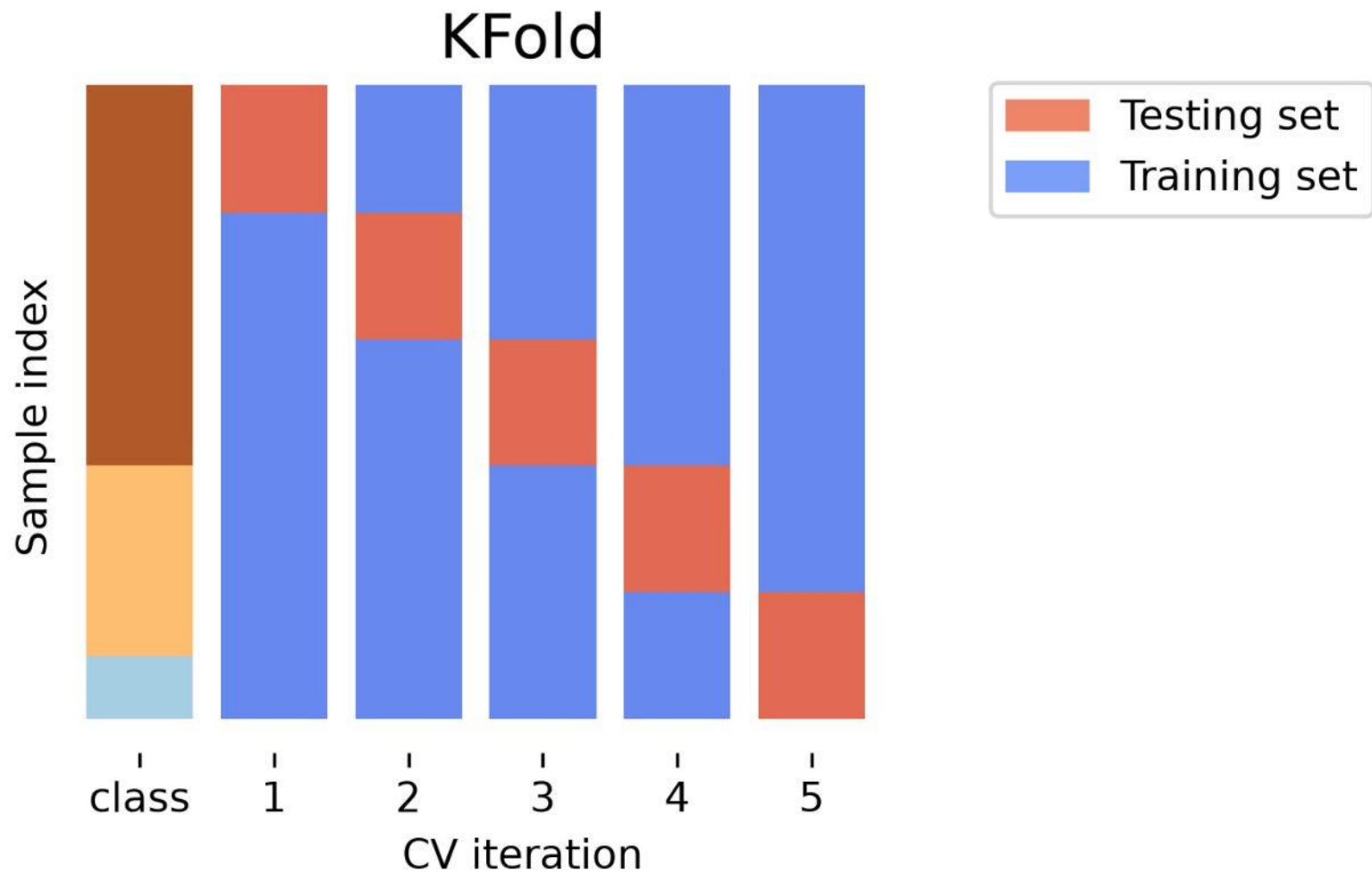
1. Use **only** the development data (`X_dev`, `y_dev`) and cross-validation to pick a kNN: `n_neighbors`, `weights`, `p`
2. **Write down your CV estimate** before looking at the held-out set
3. Evaluate once on the held-out set; the run is logged to MLflow
4. Compare: how far was your estimate off? Did the best CV score win on the held-out set?

Nested Cross-Validation

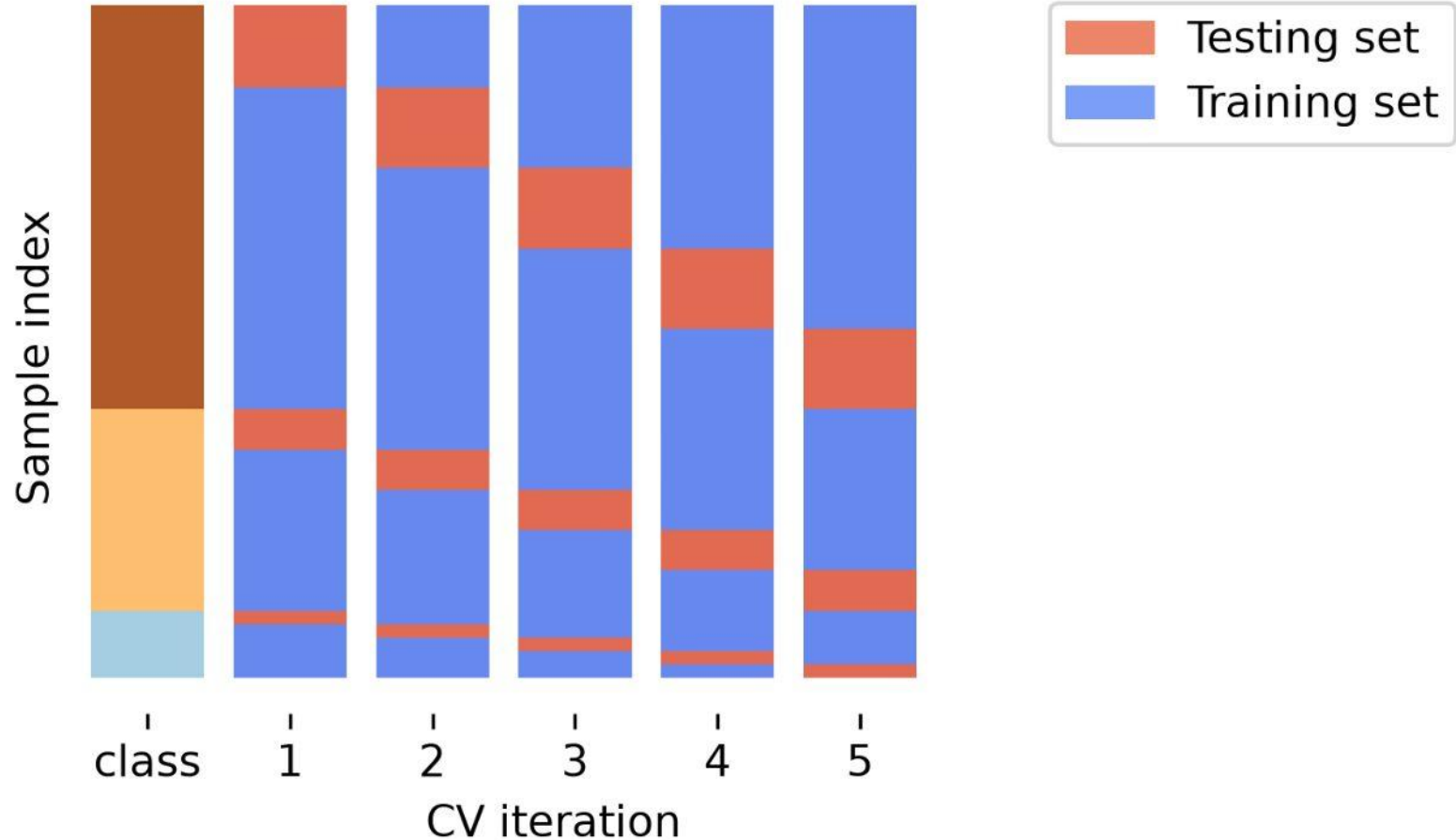
- Replace outer split by CV loop
- Doesn't yield single model (inner loop might have different best parameter settings)
- Expensive; used to estimate the performance of the whole tuning procedure

```
grid = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5)
scores = cross_val_score(grid, X, y, cv=5)  # outer loop
```

Cross-Validation Strategies



StratifiedKFold



Stratified: Ensure relative class frequencies in each fold reflect relative class frequencies on the whole dataset.

Importance of Stratification

```
y.value_counts()
```

```
0    60  
1    40
```

```
from sklearn.model_selection import cross_val_score, KFold, StratifiedKFold  
from sklearn.dummy import DummyClassifier
```

```
dc = DummyClassifier(strategy='most_frequent')  
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)  
res = cross_val_score(dc, X, y, cv=skf)  
res.mean(), res.std()
```

```
(0.6, 0.0)
```

```
kf = KFold(n_splits=5, shuffle=True, random_state=0)  
res = cross_val_score(dc, X, y, cv=kf)  
res.mean(), res.std()
```

```
(0.6, 0.138)
```

Repeated KFold and LeaveOneOut

- LeaveOneOut : `KFold(n_splits=n_samples)`

High variance, takes a long time

(see [Raschka](#) for a review and [Varoquaux](#) for empirical evaluation)

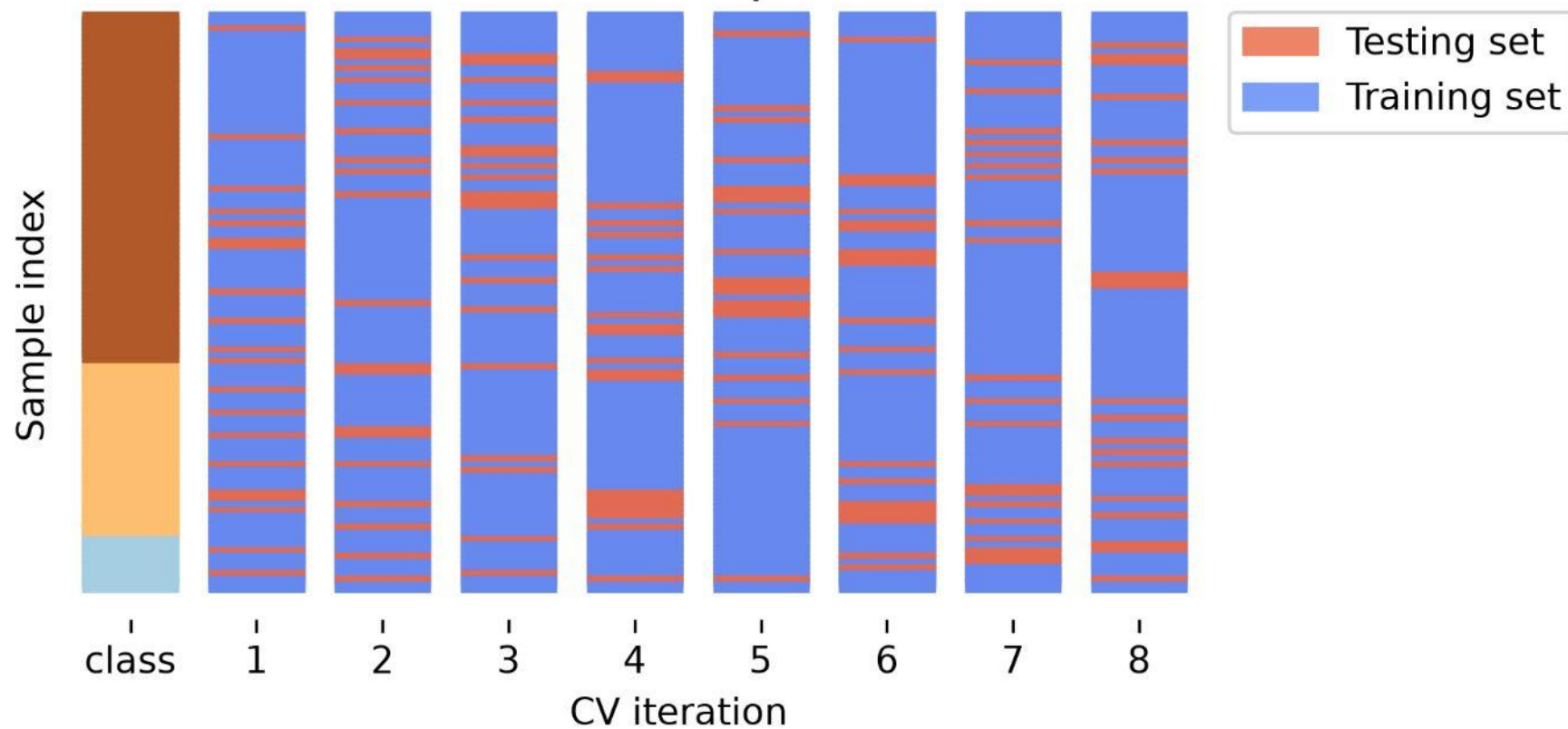
- Better: ShuffleSplit (aka Monte Carlo)

Repeatedly draw a random train/test split

- Even Better: RepeatedKFold.

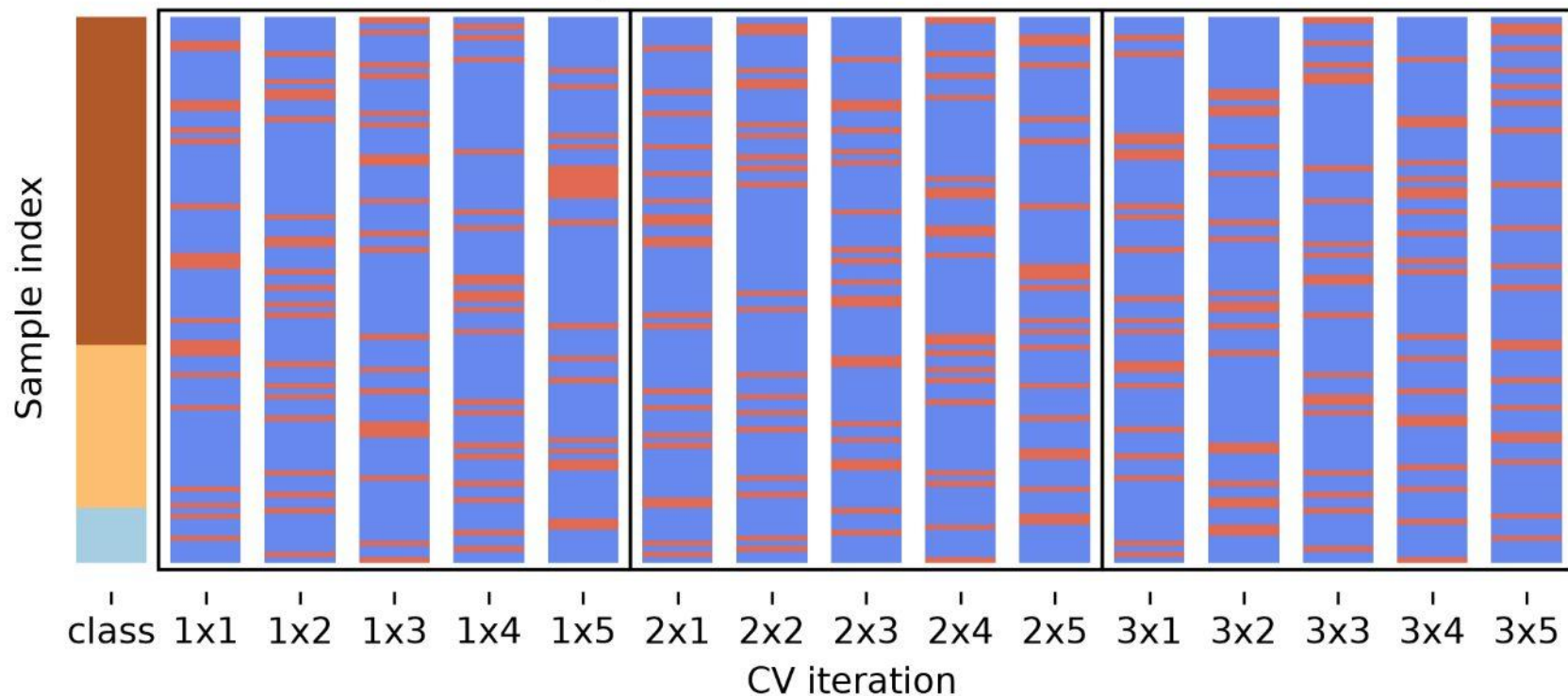
Apply KFold or StratifiedKFold multiple times with shuffled data.

ShuffleSplit



Number of iterations and test set size independent

RepeatedStratifiedKFold



Potentially less variance than StratifiedShuffleSplit.
Five times five fold or at most ten times ten fold is sufficient.

Defaults in scikit-learn

- `cv=5` by default (5-fold)
- For classifiers, integer `cv` means **stratified** k-fold
- `train_test_split` has a stratify option: `train_test_split(X, y, stratify=y)`
- **No shuffle** by default in `KFold` / `StratifiedKFold`
- Set `random_state` whenever you shuffle

Cross-Validation with non-i.i.d. data

Grouped Data

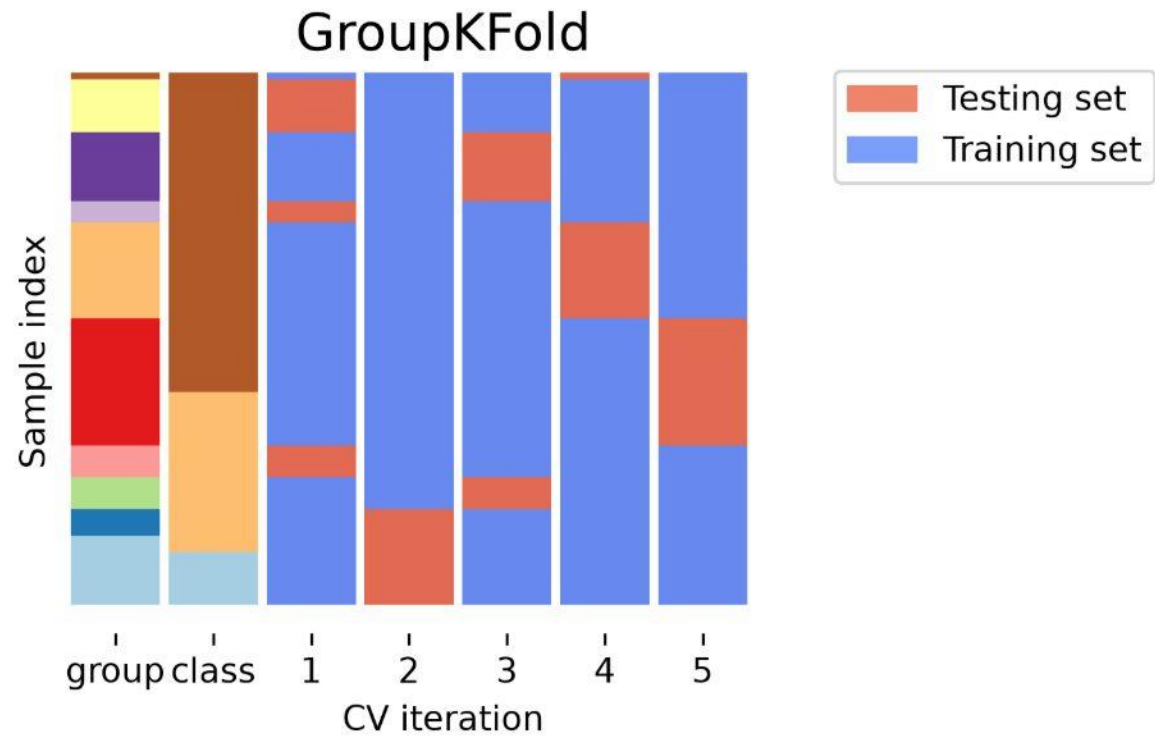
Assume we have data (medical, product, user...) from 5 cities

- Istanbul, Ankara, Izmir, Bursa, Antalya.

We can assume data within a city is more correlated than between cities.

Usage Scenarios

- Assume all future users will be in one of these cities: i.i.d.
- Assume we want to generalize to predict for a new city: not i.i.d.



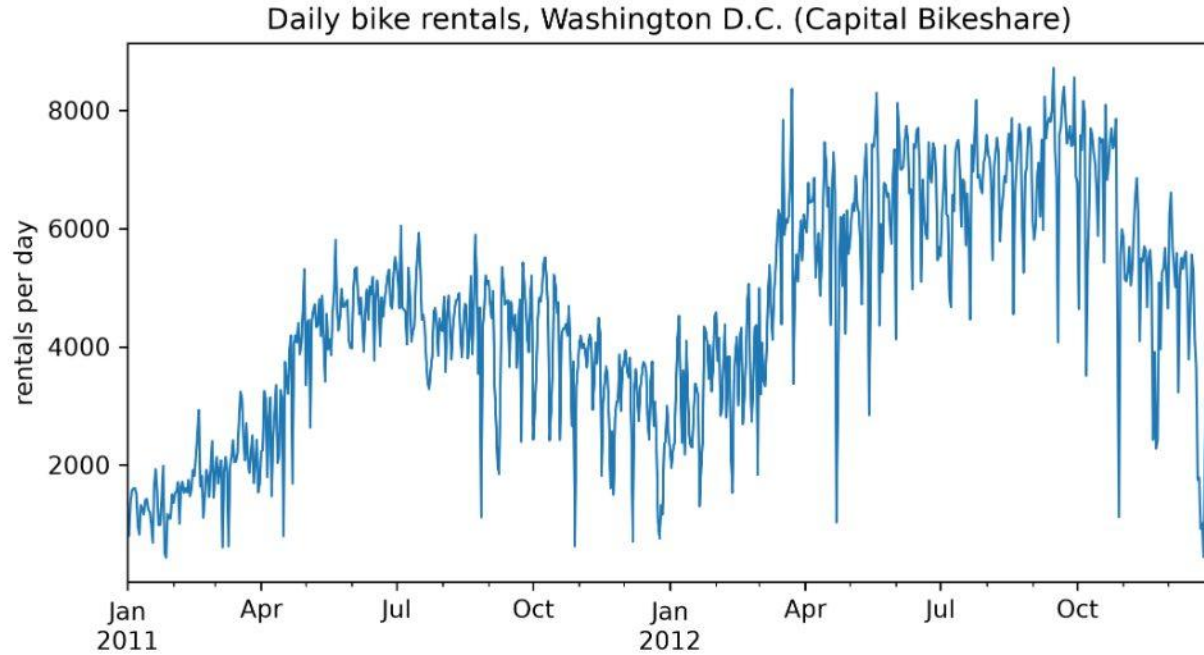
GroupKFold in Code

```
# 20 patients x 30 measurements; label = the patient's diagnosis
from sklearn.model_selection import GroupKFold, KFold, cross_val_score

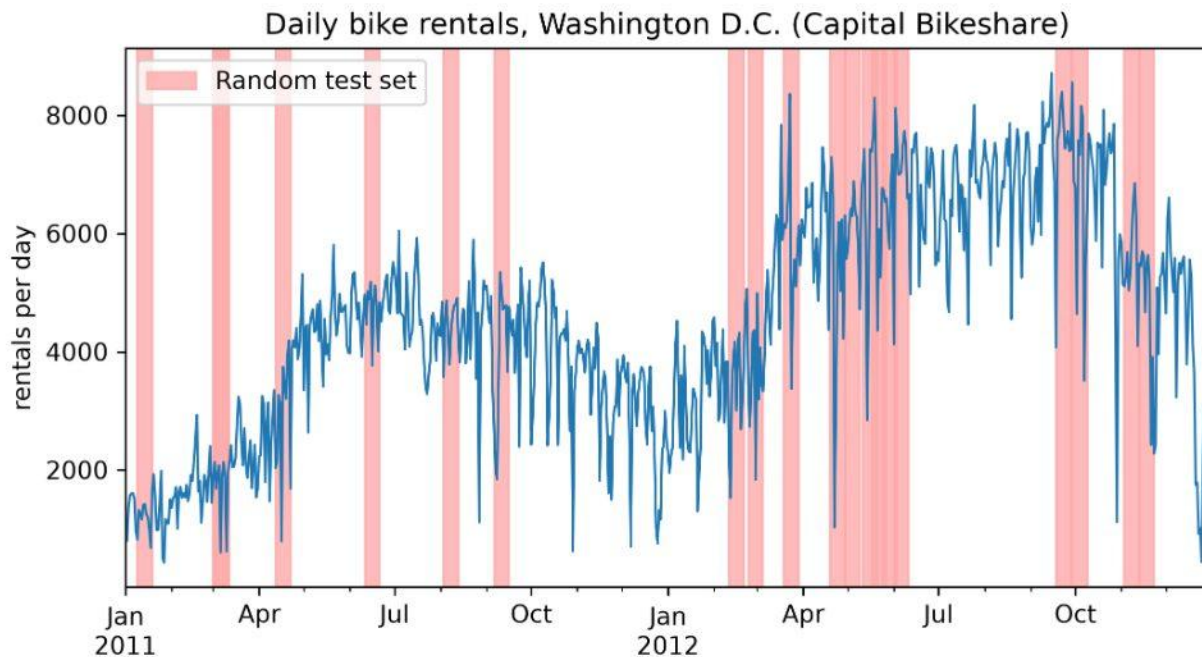
knn = KNeighborsClassifier(n_neighbors=5)
kf = cross_val_score(knn, X, y, cv=KFold(5, shuffle=True, random_state=0))
gkf = cross_val_score(knn, X, y, groups=patient_id, cv=GroupKFold(5))
print(f"KFold          accuracy: {kf.mean():.2f}")
print(f"GroupKFold accuracy: {gkf.mean():.2f}")
```

```
KFold          accuracy: 0.99
GroupKFold accuracy: 0.45
```

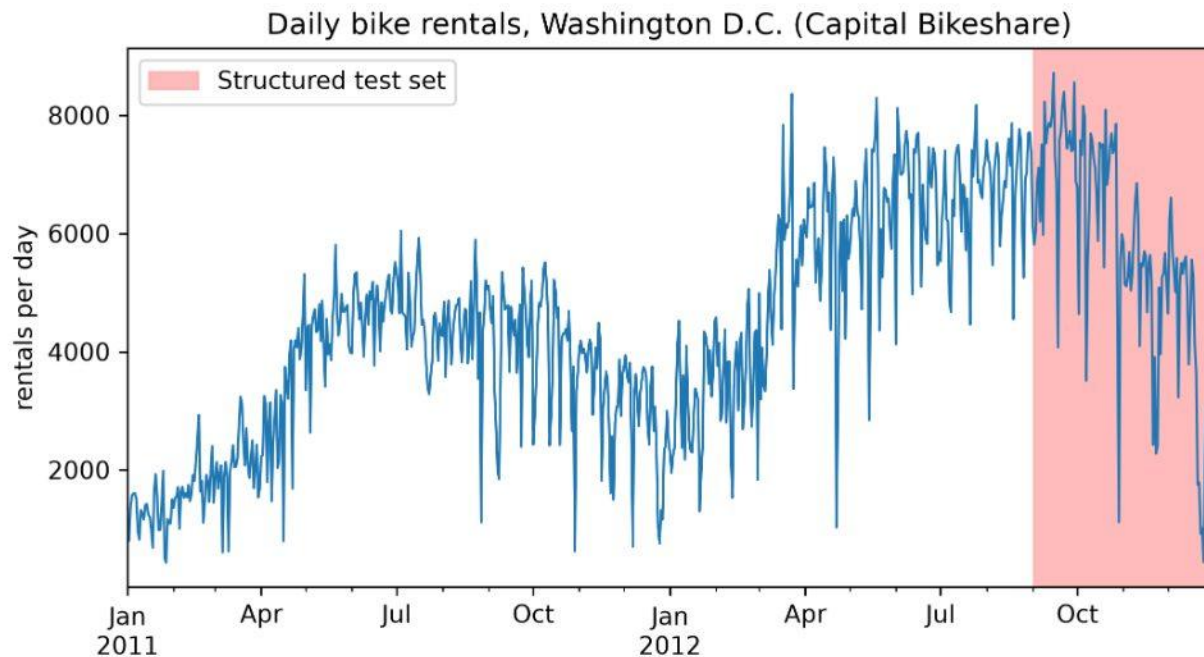
Correlations in time (and/or space)



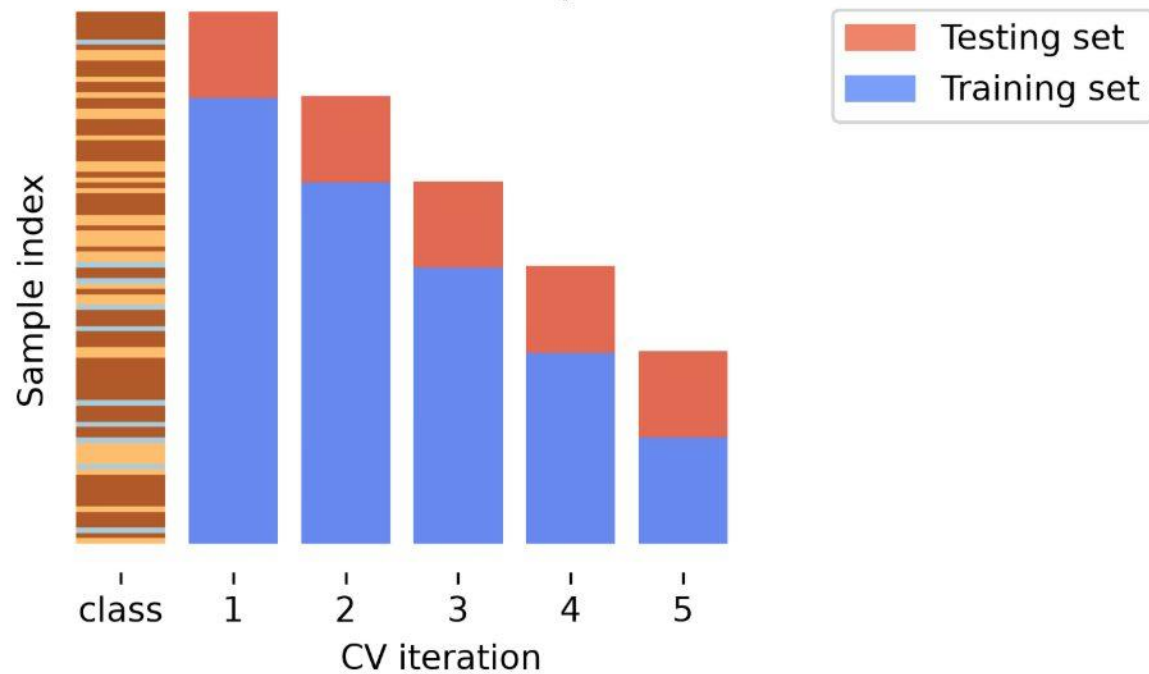
Correlations in time (and/or space)

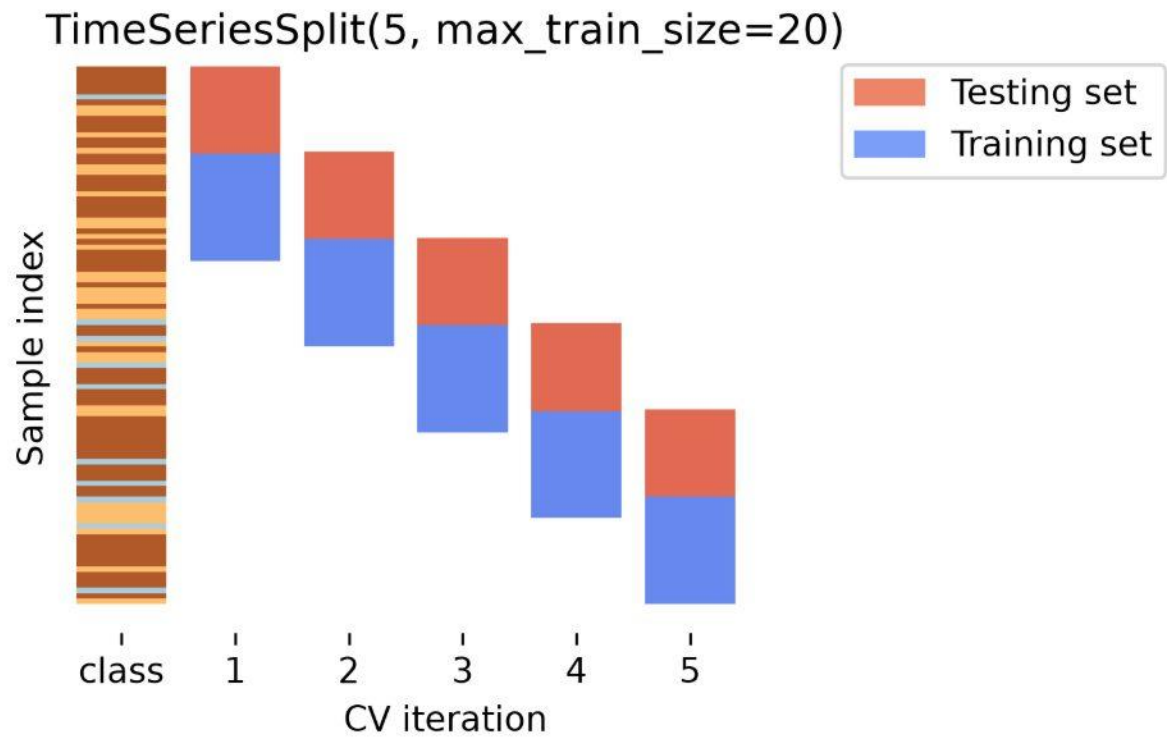


Correlations in time (and/or space)



TimeSeriesSplit





TimeSeriesSplit in Code

```
from sklearn.model_selection import TimeSeriesSplit
# bike sharing (OpenML 42712), daily totals 2011-2012
# X: rentals of the last 7 days (lag features), y: rentals today, rows sorted by date
reg = KNeighborsRegressor(n_neighbors=5)
for name, cv in [("KFold(shuffle=True)", KFold(5, shuffle=True, random_state=0)),
                 ("TimeSeriesSplit", TimeSeriesSplit(5))]:
    scores = cross_val_score(reg, X, y, cv=cv, scoring="r2")
    print(f"{name:20s} mean R^2 = {scores.mean():.2f}")
```

```
KFold(shuffle=True)    mean R^2 = 0.71
TimeSeriesSplit        mean R^2 = -0.14
```

```
TimeSeriesSplit(n_splits=5, max_train_size=None, test_size=None, gap=0)
```

Try it 2: Which Splitter? (15 min)

`lab.ipynb`, section "EXERCISE (Try it 2)": hourly bike rentals with calendar and weather features

1. **Predict first:** which gives the higher R^2 , shuffled `KFold` or `TimeSeriesSplit`, and by how much?
2. Run it. Which number would you report to the operator?
3. Add `TimeSeriesSplit(5, gap=24)` and a `max_train_size` window. What changes and why?
4. Bonus: drop the `year` feature. Which score moves more?

Using Cross-Validation Generators

```
from sklearn.model_selection import KFold, StratifiedKFold, ShuffleSplit, RepeatedStratifiedKFold
kfold = KFold(n_splits=5)
skfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
ss = ShuffleSplit(n_splits=20, train_size=.4, test_size=.3, random_state=0)
rs = RepeatedStratifiedKFold(n_splits=5, n_repeats=10, random_state=0)

print("KFold:\n", cross_val_score(KNeighborsClassifier(), X, y, cv=kfold))
print("StratifiedKFold:\n", cross_val_score(KNeighborsClassifier(), X, y, cv=skfold))
print("ShuffleSplit:\n", cross_val_score(KNeighborsClassifier(), X, y, cv=ss))
print("RepeatedStratifiedKFold:\n", cross_val_score(KNeighborsClassifier(), X, y, cv=rs))
```

```
KFold:
[0.86 0.92 0.96 0.95 0.94]
StratifiedKFold:
[0.92 0.94 0.92 0.91 0.96]
ShuffleSplit:
[0.96 0.92 0.9 0.91 0.94 0.92 0.93 0.89 0.95 0.95 0.91 0.94 0.93 0.94
0.93 0.97 0.91 0.93 0.94 0.92]
RepeatedStratifiedKFold:
[0.92 0.94 0.92 0.91 0.96 0.96 0.95 0.92 0.91 0.95 0.95 0.96 0.92 0.92
0.91 0.91 0.95 0.92 0.91 0.96 0.92 0.94 0.92 0.95 0.92 0.91 0.95 0.96
0.94 0.93 0.91 0.92 0.94 0.93 0.91 0.93 0.93 0.89 0.95 0.94 0.94 0.96
0.92 0.94 0.91 0.95 0.88 0.91 0.95 0.93]
```

cross_validate function

```
from sklearn.model_selection import cross_validate
res = cross_validate(KNeighborsClassifier(), X, y, return_train_score=True,
                    scoring=["accuracy", "roc_auc"])
res_df = pd.DataFrame(res)
res_df.round(3)
```

	fit_time	score_time	test_accuracy	train_accuracy	test_roc_auc	train_roc_auc
0	0.0	0.004	0.886	0.952	0.946	0.992
1	0.0	0.004	0.939	0.952	0.953	0.992
2	0.0	0.003	0.939	0.947	0.981	0.991
3	0.0	0.003	0.947	0.938	0.963	0.990
4	0.0	0.003	0.929	0.947	0.955	0.992

What Goes Wrong in Practice

- **Tuning on the test set:** trying "just a few" models on the test set and reporting the best
- **Reporting the best CV score** of a large search as the expected performance
- **Random splits of grouped data:** several images per patient, several sessions per user, near-duplicate rows
- **Random splits of time-ordered data:** fraud, demand, sensor data, anything that drifts
- **Preprocessing before splitting:** scaling, feature selection or imputation fitted on all data (Week 3)
- **Unstable conclusions:** one split, no `random_state`, no spread reported

Choosing a Splitter

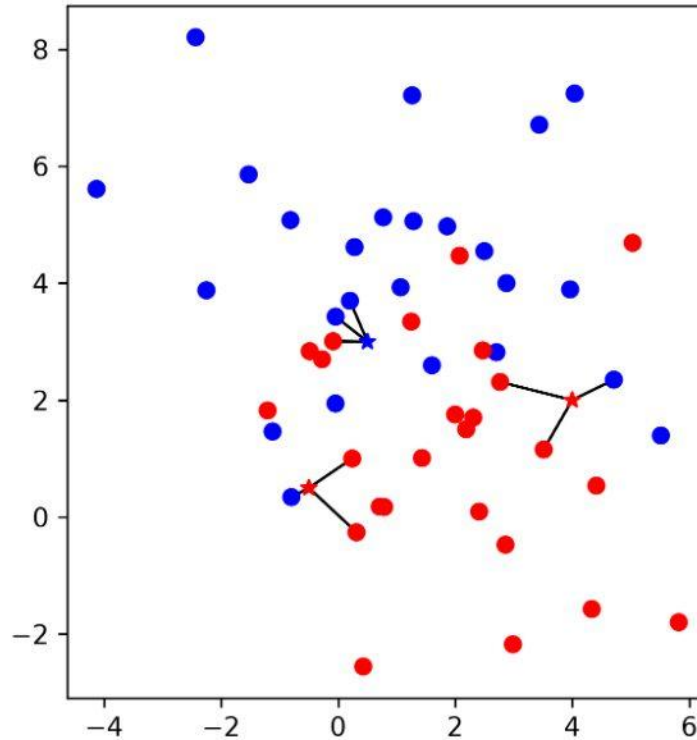
Data situation	Splitter
i.i.d. regression	KFold(shuffle=True), RepeatedKFold
i.i.d. classification, esp. imbalanced	StratifiedKFold, RepeatedStratifiedKFold
Very large data, quick estimates	ShuffleSplit, StratifiedShuffleSplit
Repeated measurements (patients, users, cities)	GroupKFold, StratifiedGroupKFold, LeaveOneGroupOut
Ordered in time, forecasting	TimeSeriesSplit (with gap, max_train_size)

Rule: the split must mimic how the model will be used.

Questions ?

Backup

Influence of Number of Neighbors



Computational Properties

Naive

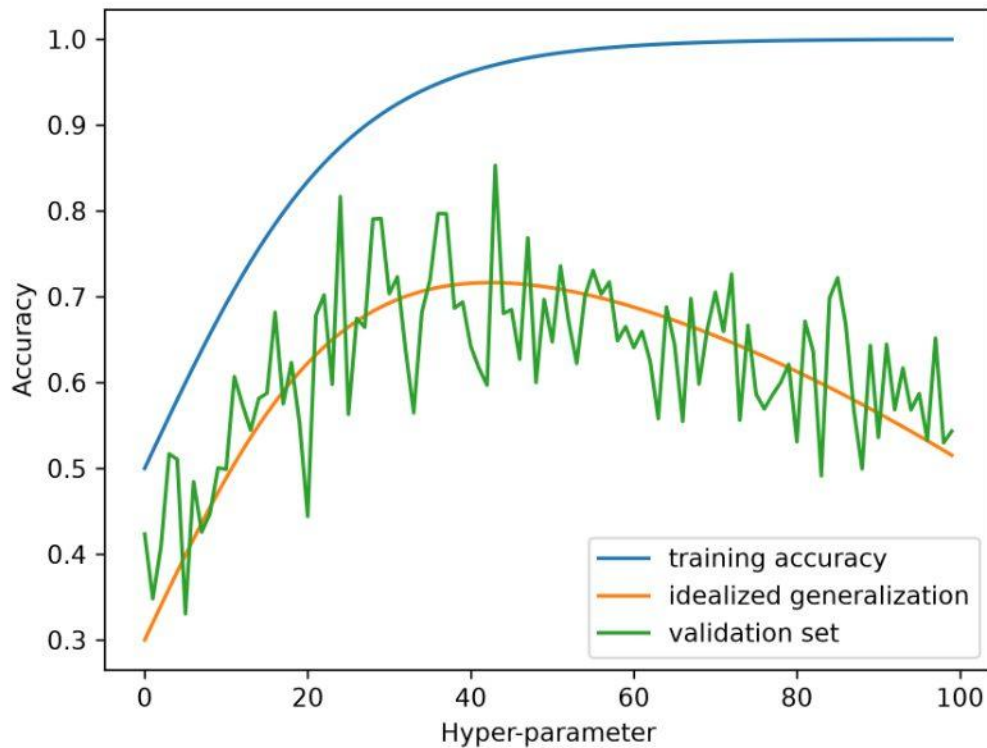
- fit: no time
- memory: $O(n * p)$
- predict: $O(n * p)$

$n=n_{\text{samples}}$ $p=n_{\text{features}}$

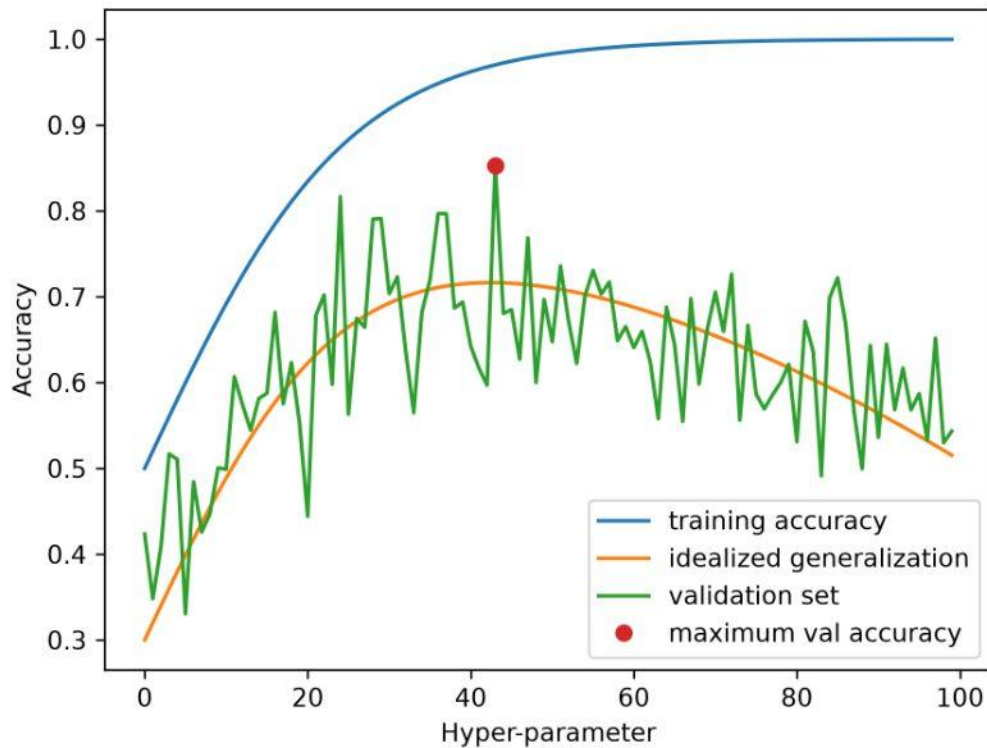
Kd-tree

- fit: $O(p * n \log n)$
 - memory: $O(n * p)$
 - predict:
 - $O(k * \log(n))$
- FOR FIXED p !

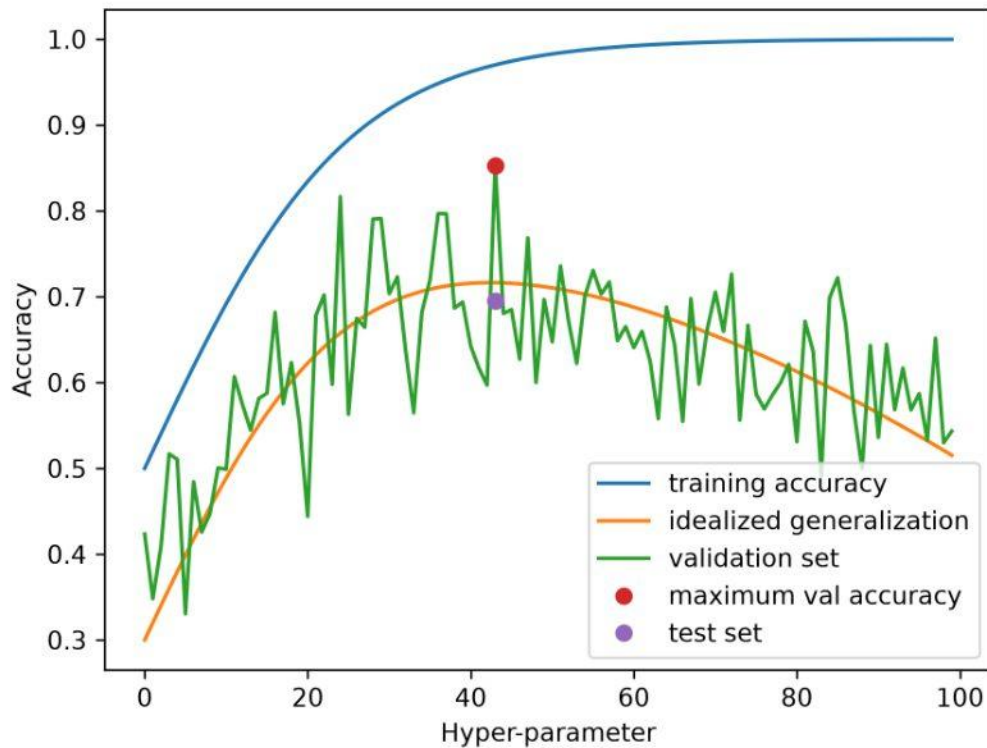
Overfitting the validation set



Overfitting the validation set



Overfitting the validation set



GridSearchCV Results

```
import pandas as pd
results = pd.DataFrame(grid.cv_results_)
results.columns
```

```
Index(['mean_fit_time', 'std_fit_time', 'mean_score_time', 'std_score_time',
       'param_n_neighbors', 'params', 'split0_test_score', 'split1_test_score',
       ...,
       'split9_test_score', 'mean_test_score', 'std_test_score', 'rank_test_score',
       'split0_train_score', ..., 'split9_train_score',
       'mean_train_score', 'std_train_score'])
```

```
results.params.head(7)
```

```
0    {'n_neighbors': 1}
1    {'n_neighbors': 3}
2    {'n_neighbors': 5}
3    {'n_neighbors': 7}
4    {'n_neighbors': 9}
5    {'n_neighbors': 11}
6    {'n_neighbors': 13}
Name: params, dtype: object
```